CS4248 Introduction to Large Language Models

Guidelines on Project Proposal

- The project proposal of each group should be at most 1 page. Please provide the following information in the proposal:
- Project group ID (e.g., G01, G02, etc.), group members' names (5), and the pre-defined project topic chosen.
- Brief description of the approach / method / algorithm to be used.
- Any additional and publicly available training data to be used? Do they require any manual annotation?
- Any external (supporting) code to be used?
- What is the uniqueness of your proposed method (if any)? How does it compare to prior methods?
    - Robustness to distractors
    - Contrastive consistency (paraphrase qn)
- Breakdown of work: The responsibility of each group member.
- Schedule/timeline of the project.

Grading Scheme

- The breakdown of the final project marks is as follows:
- Content (60%): Is the work valid and technically sound? Is the quantity of work carried out sufficient for the project? How well does the implemented system perform? Is relevant work in the literature cited and compared to?
- Presentation (30%): Is the work presented in a clear, logical, and coherent manner? (including both oral presentation and final project report)
- Novelty (10%): Does the work carried out have any novel or unique aspects?
- Your final project will be graded based on the actual work done and effort put in by your project group, and the results achieved by your group.

Robustness to distractors (Adversarial SQuAD)

The idea: Jia & Liang (2017) showed that QA models match question words rather than understanding the passage. They appended a distractor sentence: one that looks like the question but doesn't answer it.

Passage (excerpt): "…The past record was held by John Elway, who led the Broncos to victory in Super Bowl XXXIII at age 38…"
Question: "What is the name of the quarterback who was 38 in Super Bowl XXXIII?"
Added distractor: "Quarterback Jeff Dean had jersey number 37 in Champ Bowl XXXIV."
The model's answer switches from John Elway to Jeff Dean.

Across the 16 models they tested, average F1 fell from 75% to 36%.
Modern models are stronger, but they still lose a noticeable amount.
AddSent: 3,560 questions, up to 5 candidate distractors each.
AddOneSent: 1,787 questions, one distractor each.

Group ID: G17
Group Members: Chong Xin Wei, Ng Jian Yi, Soh Jun Ying Gavin, Wang Chengke, Yee Yan Hwee
Topic Chosen: Extractive QA

Approach:
A paraphrase-consistent extractor proposes the top-k answer spans; an LLM reranker picks the best one.

1. **Extractor.** Fine-tune BERT-base (baseline) and DeBERTa-v3 with a span-prediction head and sliding windows; keep each question's top 20 distinct spans.
2. **Paraphrase consistency training.** We paraphrase each training question by back-translation (English→German→English, open MarianMT models), keeping pairs with high embedding similarity. Both versions are trained on the same passage with a symmetric KL penalty between their start/end distributions over passage tokens (loss = CE(q) + CE(q′) + λ·KL), punishing different answers to the same question.
    a. Optimal transport loss (costly using hungarian algo)
    b. Tsne visualisation
3. ***Learned Span Reranker (for tiebreaking).*** *A reranker picks one of the numbered candidates using the extractor's own output representations, so its answer is always an exact passage span.*

We tune settings such as λ on 5% of the training articles, set aside for validation. The dev set is used only for final testing, where we report EM, F1 and consistency (the share of paraphrase pairs given the same answer)

Additional data:
Paraphrased training questions are generated automatically, with no manual annotation. For evaluation only: the paraphrased SQuAD test sets of Gan & Ng (2019; 1,062 non-adversarial and 56 adversarial questions from dev-v1.1) and Adversarial SQuAD (Jia & Liang, 2017).

External code:
PyTorch, Hugging Face Transformers and Datasets (its question-answering example as a preprocessing reference), MarianMT and sentence-transformers, and evaluate-v2.0.py. We write the paraphrase filtering, consistency loss, candidate generation, LLM reranker, and all evaluation code.

Uniqueness and comparison with prior work:
Beyond a standard top-1 extractor, we contribute:

- **Consistency loss, not just augmentation:** Gan & Ng (2019) add paraphrases as extra training data; we also penalise disagreement within each pair (cf. Xie et al., 2020) and ablate augmentation with and without the loss. (https://arxiv.org/abs/1904.12848)
- **Span selection, not generation:** the neural reranker selects only from the extractor's top-k spans taken directly from the passage, preventing it from generating paraphrased answers that would be penalised by Exact Match.
- **Robustness:** EM/F1 and answer agreement on paraphrased and adversarial test sets, against published BERT (Devlin et al., 2019) and DeBERTa-v3 (He et al., 2023) results.

Our central research question is therefore whether **training the extractor for paraphrase consistency and deferring final span selection to a learned span reranker** produces answers that are simultaneously more accurate and more robust than conventional top-1 extractive QA.

Breakdown of Workload:

Member	Main Responsibility
Chong Xin Wei	SQuAD preprocessing, BERT baseline and standard EM/F1 evaluation
Ng Jian Yi	DeBERTa-v3 extractor, sliding-window inference and top-(k) candidate generation
Soh Jun Ying Gavin	MarianMT paraphrase generation, similarity filtering and paraphrase dataset preparation
Wang Chengke	Paraphrase-consistency loss, training and consistency experiments
Yee Yan Hwee	Open-weight LLM reranker, adversarial evaluation, ablation studies and error analysis

Timeline:
Week 8: Consultation, Literature review, SQuAD preprocessing and BERT baseline.
Week 9: DeBERTa extractor, top-(k) span generation and paraphrase-generation pipeline.
Week 10: Implement consistency loss and train consistency-aware extractor.
Week 11: Implement and evaluate the LLM reranker; tune λ and other hyperparameters.
Week 12: Run ablation studies, paraphrase/adversarial robustness tests and error analysis.
Week 13: Final experiments, report, slides, presentation recording and code documentation.